When Does a Stride-4 Prediction Level Help a Pretrained YOLOv10 on Drone Images?

Quratulain Nayeem[1], Fahmina Taranum[2], Mohammed Mudassir Uddin[3], and Sara Rizwan[4]

[1] quratulainnayeem@gmail.com
[2] ftaranum@mjcollege.ac.in
[3] mohd.mudassiruddin7@gmail.com
[4] sara.rizwan014@gmail.com

Abstract. Detectors for drone imagery usually add a stride-4 prediction level to a COCO-pretrained network, and the gain is reported against baselines copied from other papers, with other schedules, input sizes and evaluation code. P2-Graft attaches a stride-4 branch to a pretrained YOLOv10, reading only the existing neck, so the original outputs are unchanged at the start, and initialises the branch from the pretrained stride-8 weights. The study covers VisDrone-DET2019 with YOLOv10n and YOLOv10s, input sizes from 640 to 1536 pixels, schedules of 35 to 150 epochs, and three seeds where compute allowed; every model is trained with one recipe on one NVIDIA RTX PRO 6000 Blackwell GPU and scored on test-dev, a split never used for model selection. At 640 pixels the branch raises test-dev AP from 14.09 to 15.77, a gain of 1.68 in every seed, and small-object AP by 2.07; validation AP reaches 20.64, above the published YOLOv8-N and FBRT-YOLO-N. The initialisation is worth 0.67 AP over a random start in a 35-epoch ablation, whereas matching weights by tensor name is not. The gain falls to 0.58 at 896 pixels and to −0.13 at 1280 pixels, and on YOLOv10s one seed out of three produces most of the average. The branch doubles the compute, and a baseline trained at a higher resolution is more accurate at equal GFLOPs. A stride-4 level therefore helps at small input sizes, while resolution remains the cheaper route to accuracy on VisDrone, and the initialisation of the new branch matters more than the depth of the branch.

Keywords: small object detection · UAV imagery · YOLOv10 · pretrained initialisation · VisDrone

1 Introduction

Drone images are dominated by small objects. In the VisDrone-DET2019 training split [5], 60.5% of the 343,204 annotated boxes cover less than 32^2 pixels and 26.0% less than 16^2. After the usual resize to a 640-pixel network input, the median box measures 10.7×11.4 pixels, smaller than one cell of the stride-8 grid of the finest YOLO head.

A prediction level at stride 4 (P2) is therefore the common remedy, added on top of a detector pretrained on COCO [21,19]. The extra level needs a dedicated neck path. When the neck is re-wired to feed the level, some pretrained tensors land in layers with changed inputs, and name-based weight loading copies the tensors regardless.

Gains reported for stride-4 extensions are difficult to compare. Baselines are often copied from other papers, with other schedules, input sizes and evaluation code, and the cheapest alternative, a larger input, is rarely tried. The question addressed here is narrower: when does a stride-4 level help a pretrained YOLOv10, and what does the level cost compared with raising the resolution?

The paper makes three contributions. (1) *P2-Graft*, a stride-4 branch reading only the pretrained neck, so the original outputs are unchanged at initialisation, with layers initialised from the stride-8 level (Sect. 3). (2) A controlled study of 51 training runs on one GPU with one recipe, scored from saved predictions on VisDrone test-dev, with seeds, bootstrap intervals, and baselines at the same input size and at the same GFLOPs (Sect. 4). (3) A result unfavourable to the extra level: the branch adds **1.68** AP at 640 pixels, the gain shrinks as the input grows, and a plain baseline at higher resolution is better at equal compute (Sect. 5).

2 Related Work

Small objects: Feature pyramids [10,12] are the standard answer to scale variation, and an extra high-resolution head is a frequent addition for drone images [21]. QueryDet [19] reduces the cost of

high-resolution features with sparse queries, and slicing at inference [1] trades latency for recall. Scale analyses [13] and compound scaling [14] show input size to be a strong lever for accuracy, and input size is the lever compared with the extra level below.

Label assignment: Task-aligned assignment [7] gives positives only to anchors whose centres lie inside a box. For tiny boxes the set of anchors inside the box can be empty, a gap motivating Gaussian receptive-field assignment [18] and Wasserstein-based similarity [16].

Growing a pretrained network: Net2Net [4] widens or deepens a trained network without changing the function, and ControlNet [20] attaches new branches starting as the identity. P2-Graft carries the same idea over to the detection pyramid.

3 Method

Detector: YOLOv10 [15] consists of a CSP-style backbone, a PAN neck [12] yielding the features p_3, p_4, p_5 at strides 8, 16 and 32, and one head per level with a one-to-many and a one-to-one branch; inference uses the NMS-free one-to-one branch. Boxes are regressed with a distribution focal loss [9]. Under the strict centre rule of task-aligned assignment, 16.6% of the training boxes contain no anchor centre of P3–P5 at 640 pixels, and only 3.4% contain none once a P2 level is added. The assigner of the Ultralytics release [8] softens the rule by enlarging boxes shorter than 16 pixels, and the default stays active for all models. Figure 1 shows the resulting pyramid, with the confident cells of each level on a real VisDrone crop.

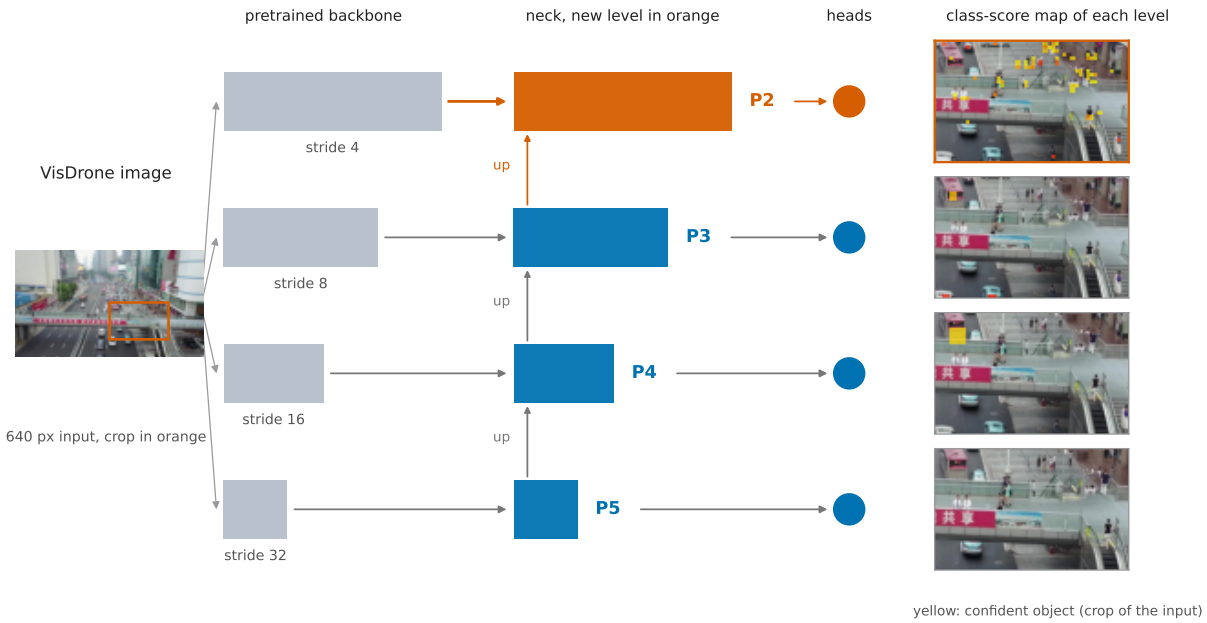


Fig. 1. Confident cells of each level on one VisDrone crop; the new stride-4 level (orange) fires on the small pedestrians.

P2-Graft: The branch extends the pyramid by one level. With b_2 the stride-4 backbone feature and p_3 the pretrained P3 neck feature, the branch computes

$$p_2 = G_\phi\big([\mathrm{Up}_{2\times}(p_3),\, b_2]\big), \qquad y_2 = H_\psi(p_2), \tag{1}$$

where G_ϕ is a neck block of the type used for P3 and H_ψ a detection head like the others. The new level is appended last, so layers and parameter names of the three original levels remain unchanged. No existing layer reads p_2 or y_2, hence the outputs on P3–P5 equal the outputs of the pretrained detector for any ϕ and ψ; on random inputs the maximum absolute difference over neck features and raw head outputs is exactly 0, for YOLOv10n and YOLOv10s. All parameters are fine-tuned together afterwards.

Warm start and depth: A level starting at random must learn what the P3 level already knows about boxes and classes. The weights of the P3 neck block and the P3 head are therefore copied into the new block and head wherever tensor shapes agree, covering 0.22 M parameters; the first 1×1 convolution of the block has half the input channels of the P3 counterpart and stays random. The deeper variant

repeats the block twice, the second bottleneck starting as a copy of the first. Both choices were fixed by a 35-epoch search over five candidates on validation AP with one seed, and the winner, the deeper block with warm start, is called P2-Graft in the remaining sections.

4 Experimental Setup

Data: VisDrone-DET2019 [5] provides 6,471 training, 548 validation and 1,610 test-dev images with ten classes; ignored regions are not used as objects. Test-dev is the primary split and was never used for a decision.

Training: Models start from the COCO checkpoint and train with SGD (learning rate 0.01 decaying linearly to 0.01×, momentum 0.937, weight decay 5×10^{-4}), three warm-up epochs, mixed precision, EMA, and a nominal batch of 64 reached by gradient accumulation. Mosaic augmentation is switched off for the last epochs. Weights of the last epoch are evaluated, with no checkpoint selection. The batch is 32 up to 640 pixels; P2-Graft uses 16 from 896 pixels and 8 at 1280 pixels because the assigner memory grows with the number of anchors, while the baseline at 896 pixels keeps 32 (Sect. 5 measures the effect). Some runs were interrupted and resumed from the last saved epoch.

Evaluation: Predictions of the one-to-one head are saved with confidence 0.001 and at most 500 boxes per image, then scored against original-resolution boxes with COCO AP [11]. Single-seed results can mislead [3], so gains are paired by seed. Intervals come from 200 image-level bootstrap resamples [6] of test-dev, shared by both models and applied to the seed-averaged AP.

Hardware: All 51 runs reported here ran on one NVIDIA RTX PRO 6000 Blackwell GPU with PyTorch 2.11 and Ultralytics 8.4.150. GFLOPs refer to the fused deploy graph.

5 Results

Results are reported on test-dev first, with validation AP alongside, and every gain is paired by seed. Table 1 lists all comparisons.

Table 1. Test-dev and validation AP (mean ± s.d. over n seeds); gain is the paired test-dev difference to the baseline.

Model	Input	n	GFLOPs	Test-dev AP	Val AP	Gain
YOLOv10n, 90 epochs						
baseline	640	3	6.6	14.09±0.09	17.75±0.26	–
P2-Graft	640	3	13.7	**15.77±0.20**	20.64±0.09	**+1.68**
baseline	896	3	13.0	18.16±0.10	23.23±0.07	–
P2-Graft (batch 16)	896	3	27.0	**18.74±0.31**	24.07±0.10	**+0.58**
P2-Graft (batch 32)	896	1	27.0	19.51	25.25	+1.31
baseline	1280	1	27.1	22.45	28.44	–
P2-Graft	1280	1	55.6	22.32	29.14	**−0.13**
baseline	1536	1	39.7	23.89	30.29	–
YOLOv10n, 150 epochs						
baseline	640	1	6.6	14.46	18.37	–
P2-Graft	640	1	13.7	**15.63**	19.86	**+1.17**
YOLOv10s, 90 epochs						
baseline	640	3	21.6	17.46±0.13	21.98±0.13	–
P2-Graft	640	3	39.8	**18.19±1.17**	23.29±1.74	**+0.73**

Same input size: At 640 pixels, P2-Graft raises test-dev AP from **14.09** to **15.77**, a gain of **+1.68** with +1.53, +1.77 and +1.75 in the three seeds (Table 1, Fig. 2a). Small-object AP (area below 32^2) rises by **2.07**, from 6.25 to 8.32, and validation AP by **2.89**. The gain shrinks at larger inputs: **+0.58** at 896 pixels (interval [+0.42, +0.79]) and **−0.13** at 1280 pixels ([−0.47, +0.22]). A control with batch 32 for the model (one seed) gives **+1.31** against the same-seed baseline and **19.51** AP instead of 19.09, so the smaller batch costs **0.42** AP of the 896-pixel result. Longer training does not favour the new level either: 150 epochs at 640 pixels give **+1.17**, below the 90-epoch gain.

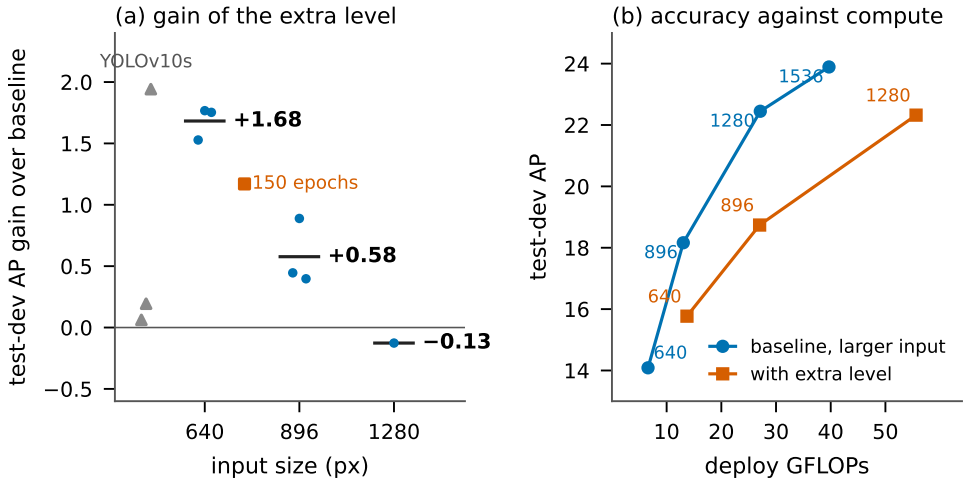


Fig. 2. (a) Test-dev AP gain by input size, one dot per seed. (b) Accuracy against compute.

Resolution against the extra level: The shrinking gain matters because P2-Graft doubles the deploy GFLOPs (13.7 against 6.6 at 640 pixels) and halves the batch-16 throughput (1,234 against 2,427 images per second at 896 pixels). Spending the same compute on resolution proves more effective. A baseline at 896 pixels costs 13.0 GFLOPs and reaches **18.16** AP, **2.39** above P2-Graft at 640 pixels. At 27 GFLOPs, the baseline at 1280 pixels reaches **22.45**, **3.71** above P2-Graft at 896 pixels (**2.94** against the batch-32 run), and the baseline keeps improving to **23.89** at 1536 pixels (Fig. 2b). Batch-1 latency, 3.2 to 4.0 ms for every model on the GPU, does not separate the models.

What matters inside the branch: Since the extra level pays off mainly at small inputs, the internals of the branch were examined at 35 epochs (Table 2, Fig. 3a). With random initialisation the level adds only **+0.26**; matching weights by tensor name gives **+0.48**; the warm start gives **+0.92**, three seeds each. Depth helps only together with the warm start (**+0.26** against **+1.05**), and a third repetition adds little (+1.16, one seed). Detail injection, a strided convolution of the stride-4 feature added to the input of the P3 head for 4% more GFLOPs, gives **+0.61** with 16 channels but only **+0.36** after 90 epochs at 640 pixels (two seeds).

Table 2. Variants at 640 pixels after 35 epochs; gain is the paired test-dev difference.

Variant	n	GFLOPs	Test-dev AP	Val AP	Gain
YOLOv10n baseline	3	6.6	13.46±0.09	16.22±0.04	–
P2 level, random init	3	12.7	13.72±0.02	17.15±0.19	+0.26
P2 level, key-matched weights	3	12.7	13.94±0.05	17.08±0.21	+0.48
P2 level, warm start	3	12.7	14.39±0.08	18.05±0.02	+0.92
2× deeper block, random init	1	13.7	13.76	17.05	+0.26
2× deeper block, warm start	3	13.7	**14.52±0.17**	18.48±0.05	**+1.05**
3× deeper block, warm start	1	14.8	14.66	18.30	+1.16
detail injection, 16 channels	3	6.9	14.08±0.08	17.16±0.21	+0.61
P2-assisted P3, 32 channels	1	8.8	14.49	17.82	+1.00

Second model scale: Whether the gain carries over to a larger model was checked on YOLOv10s at 640 pixels. The mean gain is **+0.73** (interval [+0.49, +0.96]), yet the seeds give **+0.06**, **+0.19** and **+1.94** (Fig. 3b). The standard deviation of the model across seeds is 1.17 AP against 0.13 for the baseline, so two of three seeds show no gain and the outcome is not counted as a replication.

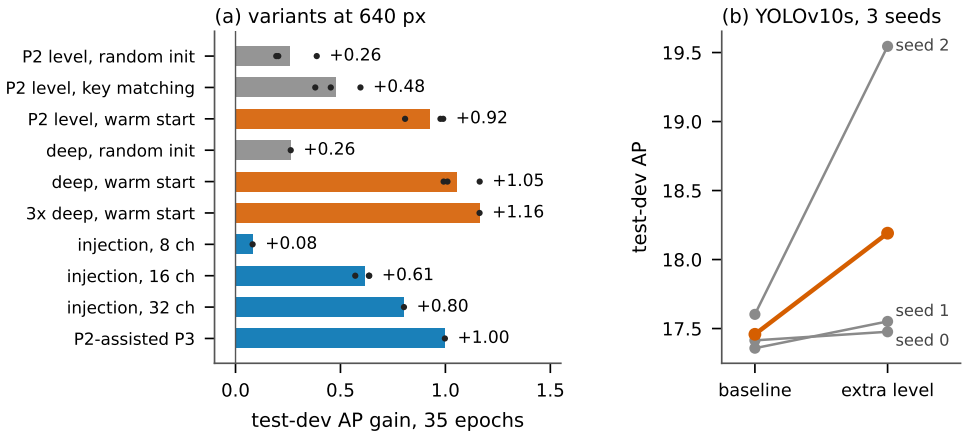


Fig. 3. (a) Test-dev AP gain of each variant at 35 epochs. (b) YOLOv10s, paired seeds.

Examples: A look at individual images complements the averages. In the validation crops of Fig. 4, chosen for the largest gain in found objects, P2-Graft finds 20 instead of 10 of 40 objects in the first crop and 21 instead of 12 of 37 in the second (score ≥ 0.3, IoU ≥ 0.5).

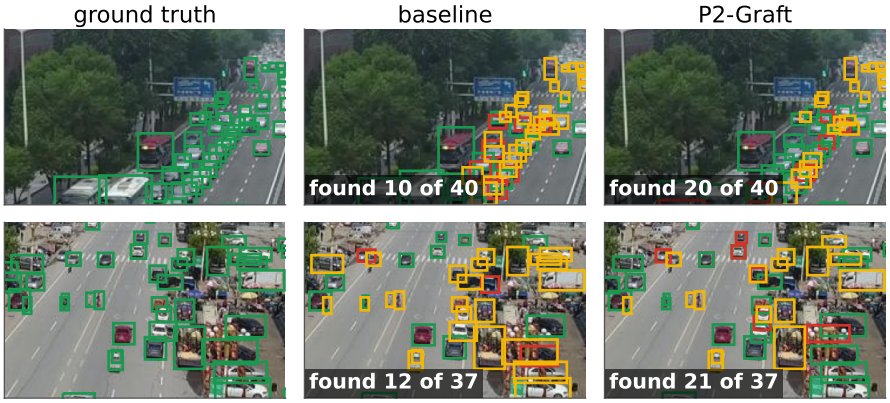


Fig. 4. Crops with the largest gain in found objects: green match, red false alarm, yellow missed.

Published results: Finally, Table 3 places the validation numbers next to results reported by other works on VisDrone. The published numbers are quoted from the respective papers, with the schedules used there (300 epochs for FBRT-YOLO), and were not re-evaluated. At 640 pixels, P2-Graft on YOLOv10n reaches **20.64** AP and **35.96** AP_{50} with 2.41 M parameters, above the reported YOLOv8-N (19.6 and 33.3, 3.2 M) and FBRT-YOLO-N (20.2 and 34.4, 0.9 M), after 90 epochs against 300. On YOLOv10s, the model reaches **23.29** and **39.86** against 23.8 and 39.3 reported for YOLOv10-S, a difference within the seed spread of 1.74 AP. The unmodified YOLOv10n at 1536 pixels reaches **30.29** and **49.42** with 2.27 M parameters, above FBRT-YOLO-M (28.4 and 45.9) at an input 2.4 times larger. DroneScan-YOLO [2] also adds a stride-4 branch and trains at 1280 pixels, reporting 35.6 and 55.3, higher than every model here; the YOLOv8s baseline reported by DroneScan-YOLO scores 23.3 and 38.7 at 640 pixels.

Table 3. Published validation results on VisDrone against the models of the study (bold, 90 epochs).

Model	Input	Params (M)	GFLOPs	AP	AP_{50}	Source
YOLOv8-N	640	3.2	8.7	19.6	33.3	[17]
FBRT-YOLO-N	640	0.9	6.7	20.2	34.4	[17]
YOLOv10-S	640	7.2	21.6	23.8	39.3	[17]
YOLOv8-S	640	11.2	28.6	23.6	39.6	[17]
FBRT-YOLO-S	640	2.9	22.9	25.9	42.4	[17]
FBRT-YOLO-M	640	7.2	58.7	28.4	45.9	[17]
DroneScan-YOLO	1280	10.23	–	35.6	55.3	[2]
YOLOv10n baseline	640	2.27	6.6	17.75	31.77	
P2-Graft, YOLOv10n	**640**	**2.41**	**13.7**	**20.64**	**35.96**	
YOLOv10s baseline	640	7.22	21.6	21.98	38.16	
P2-Graft, YOLOv10s	**640**	**7.58**	**39.8**	**23.29**	**39.86**	
P2-Graft, YOLOv10n	**1280**	**2.41**	**55.6**	**29.14**	**47.75**	
YOLOv10n baseline	**1536**	**2.27**	**39.7**	**30.29**	**49.42**	

6 Discussion

The extra level helps where objects are small relative to the grid, i.e. at low input resolution. A larger input enlarges every object for the whole network, whereas the extra level adds only a finer grid on features already computed, so at equal compute the larger input wins. The gain at 640 pixels remains worth having when the input size is fixed by the camera or the latency budget, yet the gain gives no reason to prefer the extra level when the input can be raised.

Initialisation outweighs the extra layers: with a random start the branch adds **+0.26** AP, with the warm start **+0.92**, at identical cost. Consequently, any stride-4 extension of a pretrained detector should report how the new weights are initialised.

Limitations: Only VisDrone and two model scales were used. Results at 1280 pixels, with 150 epochs, and at 1536 pixels have one seed. The 896-pixel comparison used a smaller batch for the model, and the control measuring the effect has one seed. The model was chosen on validation among five candidates, and the validation gain exceeds the test-dev gain (**+2.89** against **+1.68** at 640 pixels), so test-dev numbers are the ones to quote. Schedules of 35 to 150 epochs are shorter than the 300-epoch recipes of the literature, and absolute values should not be compared with published tables.

7 Conclusion

A stride-4 level grafted onto a pretrained YOLOv10 and initialised from the stride-8 weights gains **1.68** test-dev AP at 640 pixels. The gain shrinks with input size, does not hold reliably on YOLOv10s, and costs twice the compute. For VisDrone, raising the input resolution is the better use of compute.

Availability: Code, per-run results and trained weights:
`https://github.com/Mudassiruddin7/YOLO`
`https://huggingface.co/datasets/mudassiruddin/yolo`

References

1. Akyon, F.C., Altinuc, S.O., Temizel, A.: Slicing aided hyper inference and fine-tuning for small object detection. In: IEEE International Conference on Image Processing. pp. 966–970 (2022)
2. Bellec, Y.V.: DroneScan-YOLO: Redundancy-aware lightweight detection for tiny objects in UAV imagery. arXiv preprint arXiv:2604.13278 (2026)
3. Bouthillier, X., Delaunay, P., Bronzi, M., Trofimov, A., Nichyporuk, B., Szeto, J., et al.: Accounting for variance in machine learning benchmarks. In: Proceedings of Machine Learning and Systems (2021)
4. Chen, T., Goodfellow, I., Shlens, J.: Net2Net: Accelerating learning via knowledge transfer. In: International Conference on Learning Representations (2016)
5. Du, D., Zhu, P., Wen, L., Bian, X., Ling, H., Hu, Q., et al.: VisDrone-DET2019: The vision meets drone object detection in image challenge results. In: IEEE/CVF International Conference on Computer Vision Workshops. pp. 213–226 (2019)
6. Efron, B.: Bootstrap methods: Another look at the jackknife. The Annals of Statistics **7**(1), 1–26 (1979)

7. Feng, C., Zhong, Y., Gao, Y., Scott, M.R., Huang, W.: TOOD: Task-aligned one-stage object detection. In: IEEE/CVF International Conference on Computer Vision. pp. 3490–3499 (2021)
8. Jocher, G., Chaurasia, A., Qiu, J.: Ultralytics YOLO. Software, version 8.4.150, AGPL-3.0, `https://github.com/ultralytics/ultralytics` (2023)
9. Li, X., Wang, W., Wu, L., Chen, S., Hu, X., Li, J., Tang, J., Yang, J.: Generalized focal loss: Learning qualified and distributed bounding boxes for dense object detection. In: Advances in Neural Information Processing Systems (2020)
10. Lin, T.Y., Dollár, P., Girshick, R., He, K., Hariharan, B., Belongie, S.: Feature pyramid networks for object detection. In: IEEE Conference on Computer Vision and Pattern Recognition. pp. 936–944 (2017)
11. Lin, T.Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., Zitnick, C.L.: Microsoft COCO: Common objects in context. In: European Conference on Computer Vision. pp. 740–755 (2014)
12. Liu, S., Qi, L., Qin, H., Shi, J., Jia, J.: Path aggregation network for instance segmentation. In: IEEE Conference on Computer Vision and Pattern Recognition. pp. 8759–8768 (2018)
13. Singh, B., Davis, L.S.: An analysis of scale invariance in object detection – SNIP. In: IEEE Conference on Computer Vision and Pattern Recognition. pp. 3578–3587 (2018)
14. Tan, M., Le, Q.V.: EfficientNet: Rethinking model scaling for convolutional neural networks. In: International Conference on Machine Learning (2019)
15. Wang, A., Chen, H., Liu, L., Chen, K., Lin, Z., Han, J., Ding, G.: YOLOv10: Real-time end-to-end object detection. In: Advances in Neural Information Processing Systems. vol. 37, pp. 107984–108011 (2024)
16. Wang, J., Xu, C., Yang, W., Yu, L.: A normalized Gaussian Wasserstein distance for tiny object detection. arXiv preprint arXiv:2110.13389 (2021)
17. Xiao, Y., Xu, T., Xin, Y., Li, J.: FBRT-YOLO: Faster and better for real-time aerial image detection. In: Proceedings of the AAAI Conference on Artificial Intelligence. vol. 39, pp. 8673–8681 (2025)
18. Xu, C., Wang, J., Yang, W., Yu, H., Yu, L., Xia, G.S.: RFLA: Gaussian receptive field based label assignment for tiny object detection. In: European Conference on Computer Vision. pp. 526–543 (2022)
19. Yang, C., Huang, Z., Wang, N.: QueryDet: Cascaded sparse query for accelerating high-resolution small object detection. In: IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 13658–13667 (2022)
20. Zhang, L., Rao, A., Agrawala, M.: Adding conditional control to text-to-image diffusion models. In: IEEE/CVF International Conference on Computer Vision. pp. 3813–3824 (2023)
21. Zhu, X., Lyu, S., Wang, X., Zhao, Q.: TPH-YOLOv5: Improved YOLOv5 based on transformer prediction head for object detection on drone-captured scenarios. In: IEEE/CVF International Conference on Computer Vision Workshops. pp. 2778–2788 (2021)